Regularization and polynomial regression

CSCI-P 556

ZORAN TIGANJ

Today

- Polynomial regression
- Regularization (Lass, Ridge, Elastic net, Early stopping)

Examples of linear models

Linear models have a linear relationship between dependent variable (y) and parameters

Simple Linear Regression

$$y = \beta_0 + \beta_1 x + \epsilon$$

Description: This model predicts a response based on a single predictor variable.
Example Application: Predicting the salary of an employee based on years of experience.

Multiple Linear Regression

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon$$

Description: This model uses multiple predictor variables to predict a response.
Example Application: Estimating the price of a house based on its size, age, and location.

Polynomial Regression

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_n x^n + \epsilon$$

Description: Although it models non-linear relationships, it is considered a linear model because it is linear in the parameters.
Example Application: Modeling the growth rate of bacteria depending on the nutrient concentration

Examples of non-linear models

Non-Linear models have a non-linear relationship between dependent variable (y or p) and parameters

Logistic Regression

$$p = \frac{1}{1+e^{-(\beta_0+\beta_1 x)}}$$

Description: Despite its name, logistic regression is a non-linear model used for binary classification.
Example Application: Predicting whether a patient has a disease (1) or not (0) based on their test results.

Exponential Growth Model

$$y = a \cdot e^{bx}$$

Description used for growth processes where the increase is proportional to the current amount.
Example Application: Predicting population growth.

Function fitting

$$y = a\sin(bx + c) + d$$

Description: Fitting some known functional relationship. E.g., sinusoidal is appropriate for data exhibiting periodic fluctuations.
Example Application: Seasonal variations in temperature or other cyclic phenomena.

Why does the difference between linear and non-linear models matter?

- Linear models will usually have a closed-form solution in the form of a Normal Equation and the loss function is convex (if we use MSE as a loss function, also called Ordinary Least Squares method).
  - Closed closed-form solution might not exist if we can't compute the inverse in the normal equation

$$\mathbf{\Theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

- Non-linear models can represent more complex relationships. This ability makes them suitable for dealing with real-world phenomena that inherently exhibit non-linear dynamics, such as exponential growth, saturation effects, and threshold effects.
  - Non-linear models typically require iterative, numerical approaches, such as gradient descent, which can be computationally intensive and require more data to achieve stable estimates (loss function often not convex)

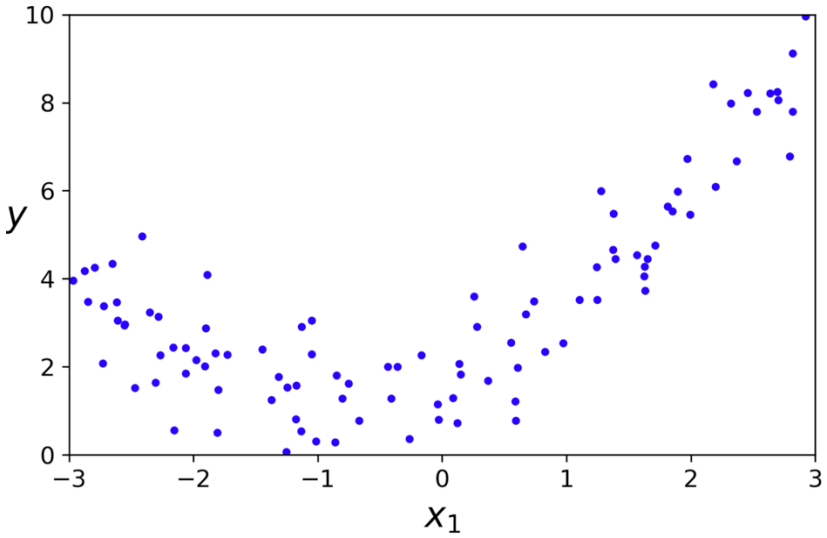
Polynomial Regression

- What if your data is more complex than a straight line?
- Surprisingly, you can use a linear model to fit nonlinear data.
- A simple way to do this is to add powers of each feature as new features, then train a linear model on this extended set of features.
- This technique is called Polynomial Regression

Polynomial Regression

- Let's generate some nonlinear data, based on a simple quadratic equation:

```
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(m, 1)
```



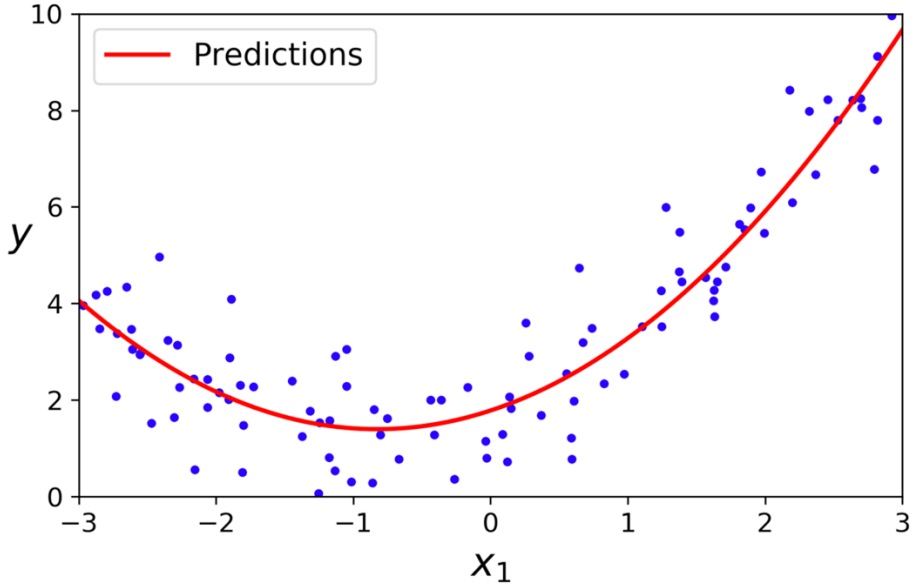
Polynomial Regression

- Let's use ScikitLearn's PolynomialFeatures class to transform our training data, adding the square (second-degree polynomial) of each feature in the training set as a new feature (in this case there is just one feature):

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929, 0.56664654])
```

```
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

Polynomial Regression



$$y = 0.5{x_1}^2 + 1.0x_1 + 2.0 + \text{Gaussian noise}$$

$$\hat{y} = 0.56{x_1}^2 + 0.93x_1 + 1.78$$

Polynomial Regression

- Note that when there are multiple features, Polynomial Regression is capable of finding relationships between features.
- This is made possible by the fact that PolynomialFeatures also adds all combinations of features up to the given degree.
  - For example, if there were two features a and b, PolynomialFeatures with degree=3 would not only add the features a^2, a^3, b^2, and b^3, but also the combinations ab, a^2b, and ab^2.

WARNING

`PolynomialFeatures(degree=d)` transforms an array containing n features into an array containing $\frac{(n+d)!}{d!\,n!}$ features, where $n!$ is the *factorial* of n, equal to $1 \times 2 \times 3 \times \cdots \times n$. Beware of the combinatorial explosion of the number of features!

Learning Curves

- If you perform high-degree Polynomial Regression, you will likely fit the training data much better than with plain Linear Regression.
- This high-degree Polynomial Regression model is severely overfitting the training data, while the linear model is underfitting it.
- How can you tell that your model is overfitting or underfitting the data?

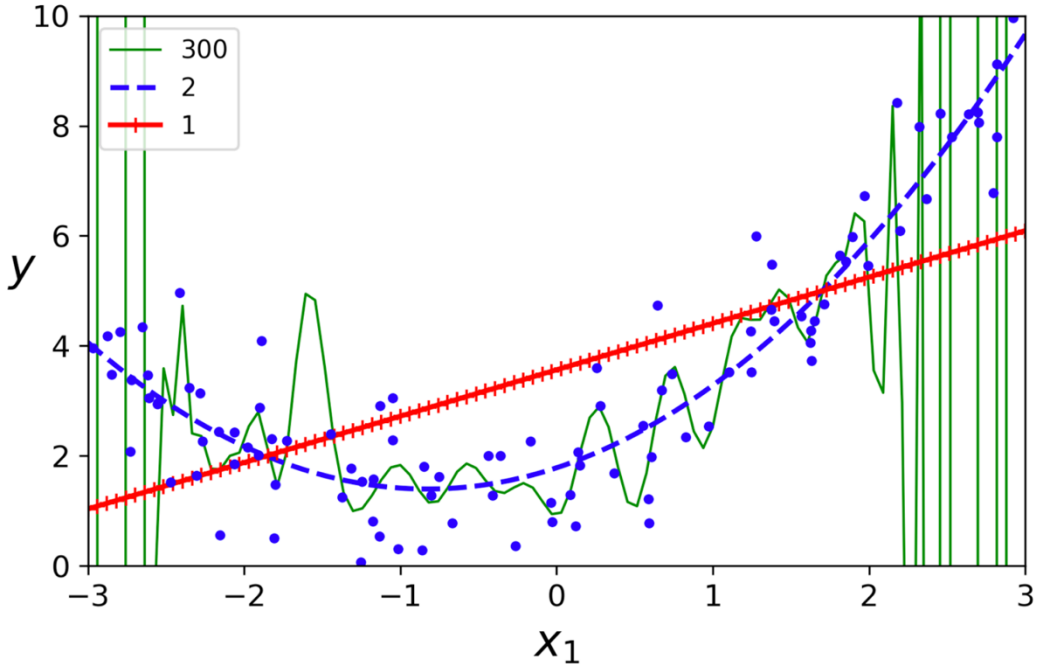
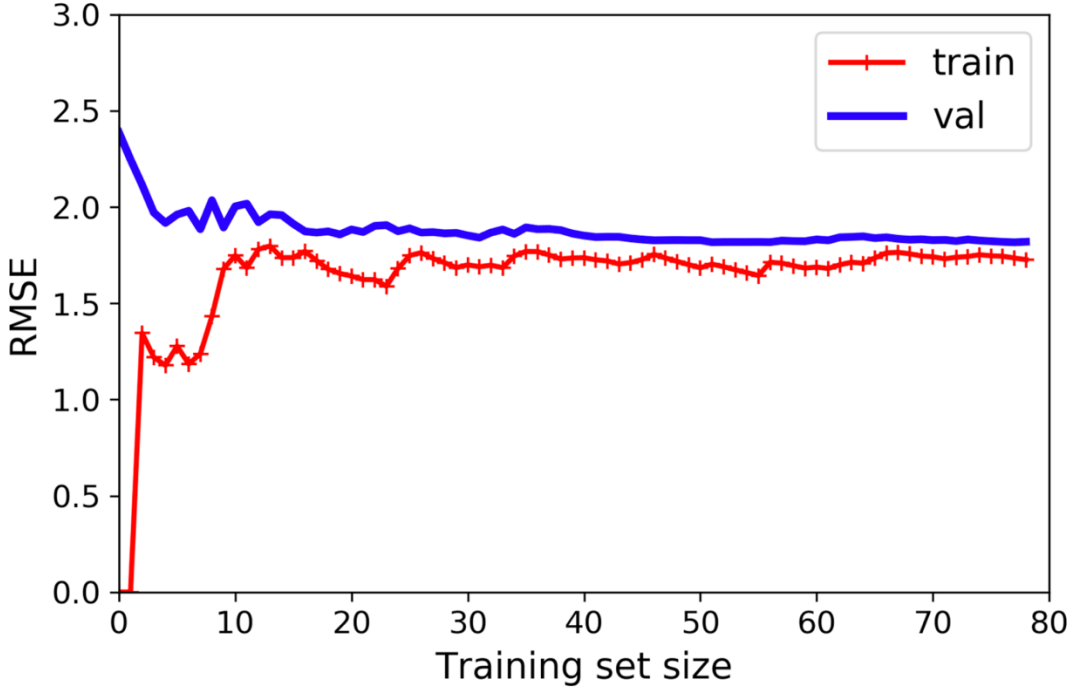


Figure 4-14. High-degree Polynomial Regression

Learning Curves: Underfitting

- Linear model: These learning curves are typical of a model that's underfitting. Both curves have reached a plateau; they are close and fairly high.



Learning Curves: Overfitting

- 10th degree polynomial:
  - The error on the training data is much lower than with the Linear Regression model.
  - There is a gap between the curves. This means that the model performs significantly better on the training data than on the validation data, which is the hallmark of an overfitting model.
  - If you used a much larger training set, however, the two curves would continue to get closer.

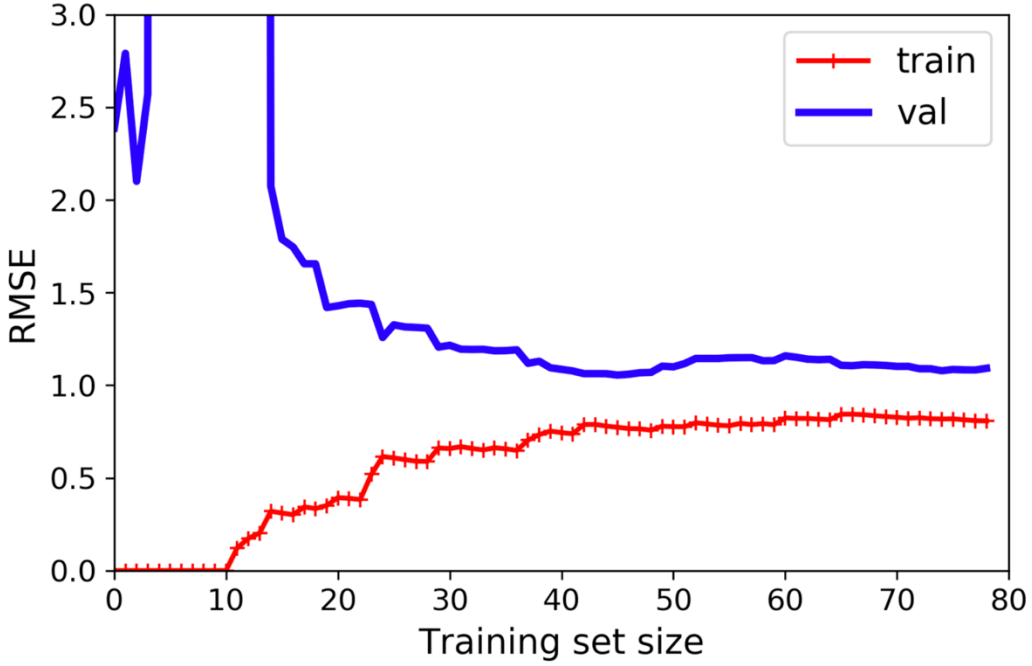


Figure 4-16. Learning curves for the 10th-degree polynomial model

THE BIAS/VARIANCE TRADE-OFF

An important theoretical result of statistics and Machine Learning is the fact that a model's generalization error can be expressed as the sum of three very different errors:

Bias

This part of the generalization error is due to wrong assumptions, such as assuming that the data is linear when it is actually quadratic. A high-bias model is most likely to underfit the training data.[8]

Variance

This part is due to the model's excessive sensitivity to small variations in the training data. A model with many degrees of freedom (such as a high-degree polynomial model) is likely to have high variance and thus overfit the training data.

Irreducible error

This part is due to the noisiness of the data itself. The only way to reduce this part of the error is to clean up the data (e.g., fix the data sources, such as broken sensors, or detect and remove outliers).

Increasing a model's complexity will typically increase its variance and reduce its bias. Conversely, reducing a model's complexity increases its bias and reduces its variance. This is why it is called a trade-off.

Regularized Linear Models

- A good way to reduce overfitting is to regularize the model (i.e., to constrain it): the fewer degrees of freedom it has, the harder it will be for it to overfit the data.
- A simple way to regularize a polynomial model is to reduce the number of polynomial degrees.
- For a linear model, regularization is typically achieved by constraining the weights of the model.
- We will now look at three different ways to constrain the weights:
  - Ridge Regression,
  - Lasso Regression,
  - Elastic Net

Ridge Regression

- Ridge Regression (also called Tikhonov regularization) is a regularized version of Linear Regression: a regularization term added to the cost function is

$$\alpha \sum_{i=1}^{n} {\theta_i}^2$$

- This forces the learning algorithm to not only fit the data but also keep the model weights as small as possible.
- Loss function with regularization:

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \sum_{i=1}^{n} {\theta_i}^2$$

Ridge Regression

- Ridge Regression (also called Tikhonov regularization) is a regularized version of Linear Regression: a regularization term added to the cost function is

$$\alpha \sum_{i=1}^{n} {\theta_i}^2$$

- This forces the learning algorithm to not only fit the data but also keep the model weights as small as possible.
- Loss function with regularization:

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \sum_{i=1}^{n} {\theta_i}^2$$

- If α is very large, then all weights end up very close to zero and the result is a flat line going through the data's mean (note that sum starts from 1).

Ridge Regression

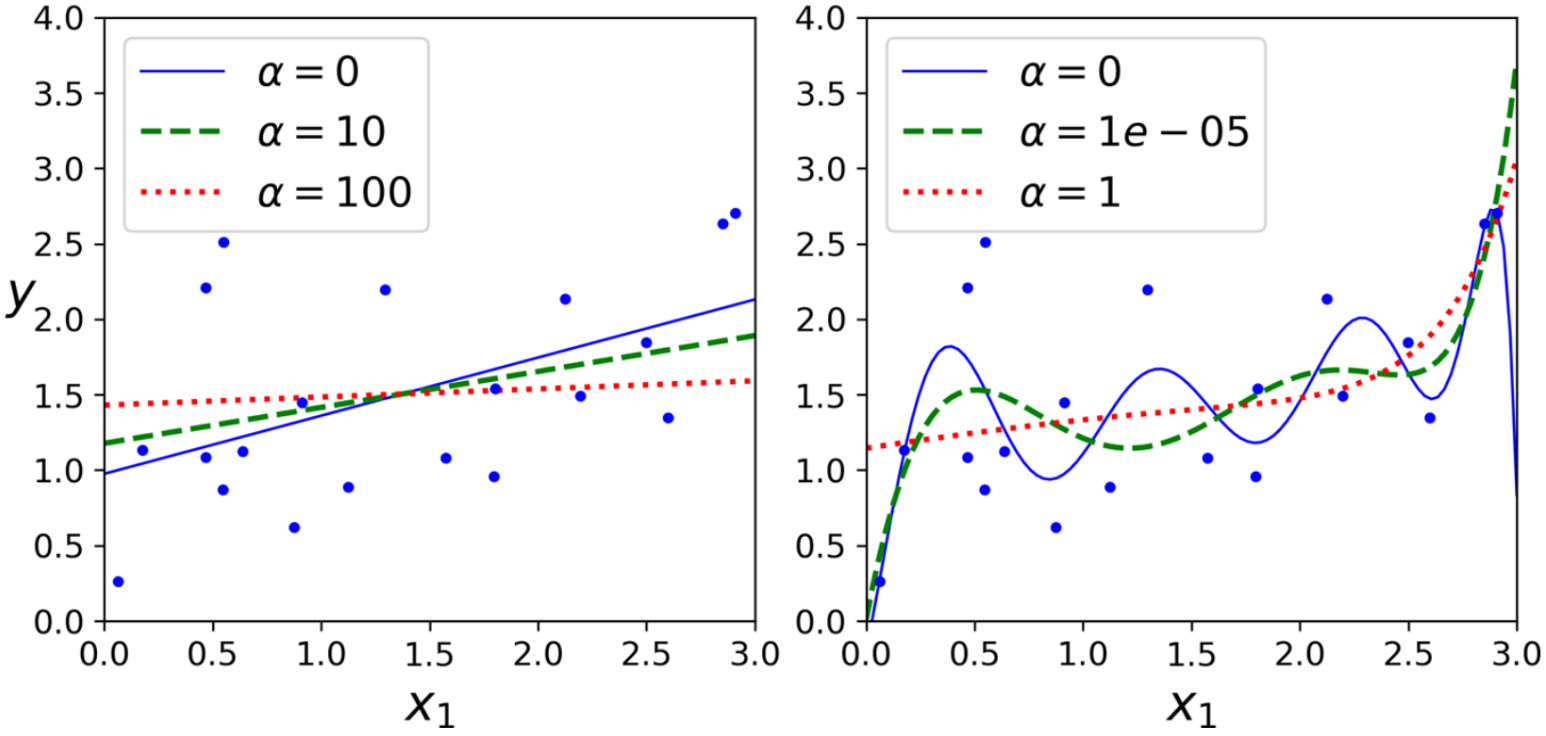


Figure 4-17. A linear model (left) and a polynomial model (right), both with various levels of Ridge regularization

Ridge Regression

- Ridge Regression closed-form solution

$$\widehat{\boldsymbol{\theta}} = \left(\mathbf{X}^\intercal \mathbf{X} + \alpha \mathbf{A}\right)^{-1} \mathbf{X}^\intercal \mathbf{y}$$

```
np.random.seed(42)
m = 20
X = 3 * np.random.rand(m, 1)
y = 1 + 0.5 * X + np.random.randn(m, 1) / 1.5
X_new = np.linspace(0, 3, 100).reshape(100, 1)
```

```
from sklearn.linear_model import Ridge
ridge_reg = Ridge(alpha=1, solver="cholesky", random_state=42)
ridge_reg.fit(X, y)
ridge_reg.predict([[1.5]])
```

```
array([[1.55071465]])
```

Ridge Regression

- And using Stochastic Gradient Descent:

```
>>> sgd_reg = SGDRegressor(penalty="l2")
>>> sgd_reg.fit(X, y.ravel())
>>> sgd_reg.predict([[1.5]])
array([1.47012588])
```

The `penalty` hyperparameter sets the type of regularization term to use. Specifying `"l2"` indicates that you want SGD to add a regularization term to the cost function equal to half the square of the ℓ_2 norm of the weight vector: this is simply Ridge Regression.

Lasso Regression

- Least Absolute Shrinkage and Selection Operator Regression (usually simply called Lasso Regression) is another regularized version of Linear Regression.
- Just like Ridge Regression, it adds a regularization term to the cost function, but it uses the ℓ_1 norm of the weight vector instead of half the square of the ℓ_2 norm.

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \alpha \sum_{i=1}^{n} |\theta_i|$$

Lasso Regression

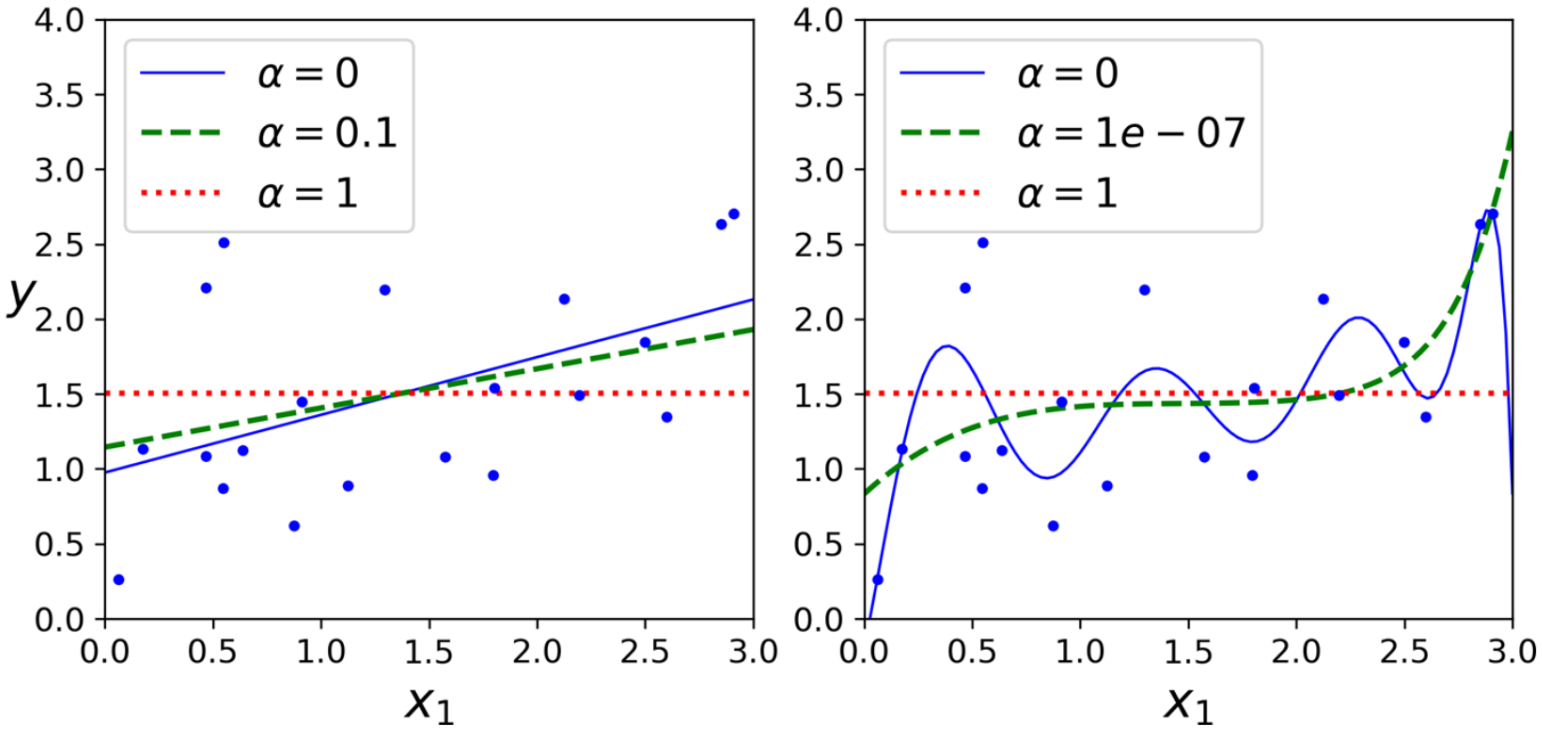


Figure 4-18. A linear model (left) and a polynomial model (right), both using various levels of Lasso regularization

Lasso Regression

- An important characteristic of Lasso Regression is that it tends to eliminate the weights of the least important features (i.e.,set them to zero).
- In other words, Lasso Regression automatically performs feature selection and out puts a sparse model (i.e., with few nonzero feature weights).

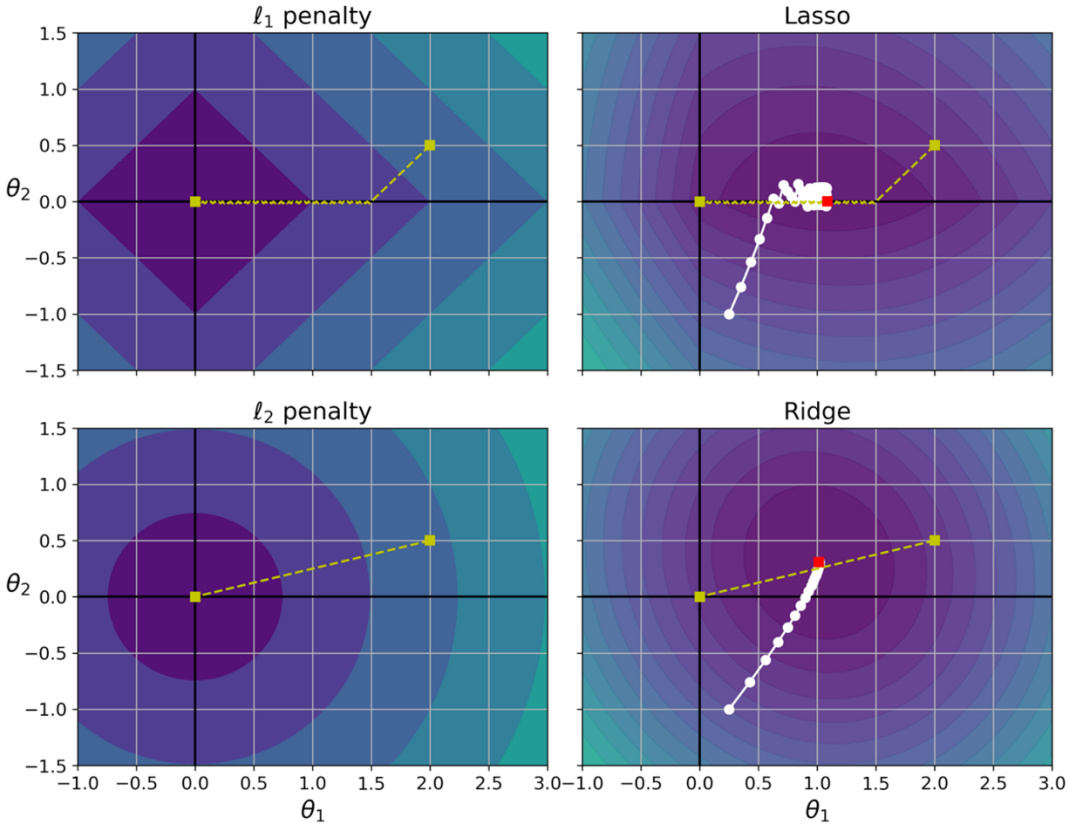


Figure 4-19. Lasso versus Ridge regularization

https://explained.ai/regularization/index.html

Lasso Regression

Here is a small Scikit-Learn example using the `Lasso` class:

```
>>> from sklearn.linear_model import Lasso
>>> lasso_reg = Lasso(alpha=0.1)
>>> lasso_reg.fit(X, y)
>>> lasso_reg.predict([[1.5]])
array([1.53788174])
```

Note that you could instead use `SGDRegressor(penalty="l1")`.

Elastic Net

- Elastic Net is a middle ground be tween Ridge Regression and Lasso Regression.
- The regularization term is a simple mix of both Ridge and Lasso's regularization terms, and you can control the mix ratio r.
  - when r = 0, Elastic Net is equivalent to Ridge Regression,
  - when r = 1, it is equivalent to Lasso Regression

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + r\alpha \sum_{i=1}^{n} |\theta_i| + \frac{1-r}{2}\alpha \sum_{i=1}^{n} {\theta_i}^2$$

Here is a short example that uses Scikit-Learn's `ElasticNet` (`l1_ratio` corresponds to the mix ratio *r*):

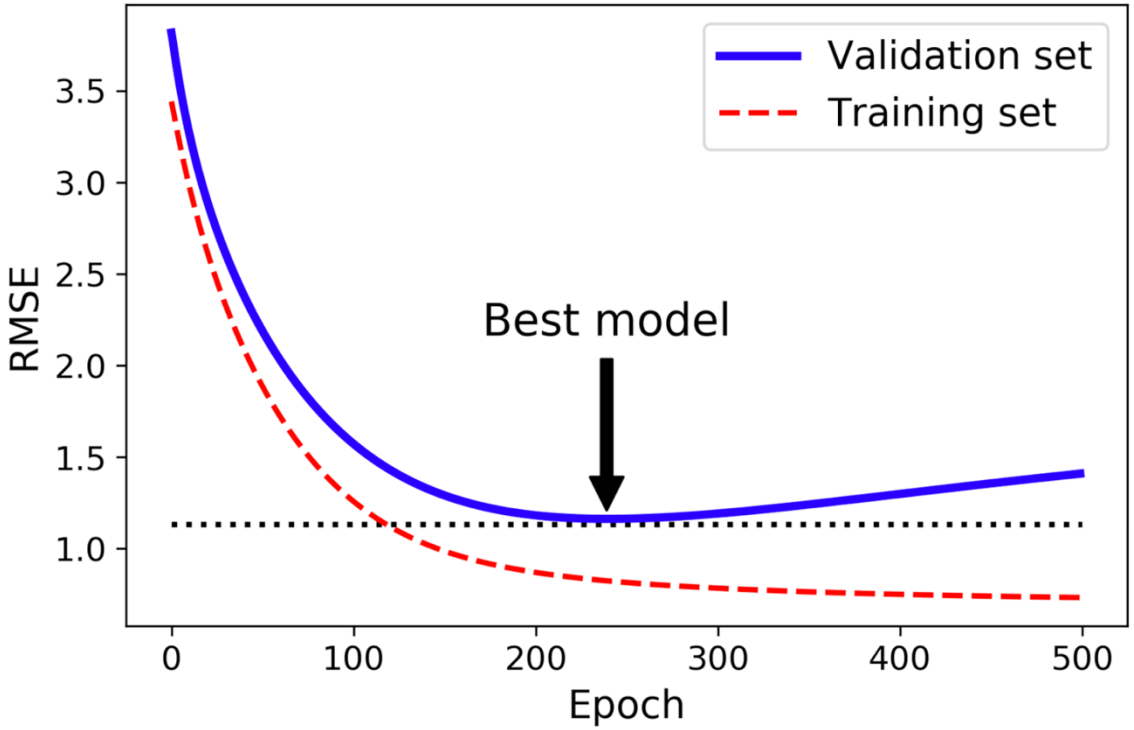
```
>>> from sklearn.linear_model import ElasticNet
>>> elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
>>> elastic_net.fit(X, y)
>>> elastic_net.predict([[1.5]])
array([1.54333232])
```

What regularization those choose

- So when should you use plain Linear Regression (i.e., without any regularization), Ridge, Lasso, or Elastic Net?
  - It is almost always preferable to have at least a little bit of regularization, so generally you should avoid plain Linear Regression.
  - Ridge is a good default, but if you suspect that only a few features are useful, you should prefer Lasso or Elastic Net because they tend to reduce the useless features' weights down to zero.
  - In general, Elastic Net is preferred over Lasso because Lasso may behave erratically when the number of features is greater than the number of training instances or when several features are strongly correlated.

Early Stopping

- A very different way to regularize iterative learning algorithms such as Gradient Descent is to stop training as soon as the validation error reaches a minimum.
- This is called early stopping.



Early Stopping

- With Stochastic and Mini-batch Gradient Descent, the curves are not so smooth, and it may be hard to know whether you have reached the minimum or not.
- One solution is to stop only after the validation error has been above the minimum for some time (when you are confident that the model will not do any better), then roll back the model parameters to the point where the validation error was at a minimum.

Next time

- Logistic regression, from Chapter 4 from *Hands on machine learning textbook*